Tail Wakeup Latency on Intel Hybrid CPUs: Evaluating Linux CPU Placement and Wakeup Placement Under Mixed Load

Shreyas Vattikuti

ABSTRACT

Hybrid Intel processors expose performance (P) and efficiency (E) cores to one Linux scheduler, and interactive tasks on such machines must survive periods when batch work saturates the P-cores. Intel's Thread Director supplies core-type hints, but Linux still chooses the final CPU in the wakeup path. We measure wakeup tail latency for an eight-worker schbench probe while twelve sysbench threads occupy all P-core SMT siblings on an Intel Core i7-13650HX running Linux 7.1.8 with EEVDF. In 30 paired runs, we compare floating affinity (CPUs 0–17) with pinning to a six-E-core test pool (CPUs 12–17). The distribution body is unchanged: median wakeup latency is 43 µs floating and 44 µs pinned. The per-run p99.9, which is effectively the maximum with 120 samples, diverges: median 162 µs floating versus 701 µs pinned, while the worst cases are 2836 µs and 2852 µs. Pinning moves mass into the 250–1000 µs band but does not remove multi-millisecond events: 5/30 floating and 4/30 pinned runs exceed 1 ms. A paired test does not establish a reliable ranking (Wilcoxon $p = 0.17$), because the distributions cross. A separate 30 s trace over 707 wakeups shows that all nine wakeups slower than 100 µs hit an occupied target CPU; four were placement races, including one pair 4 µs apart. Blanket E-core pinning trades a rare cross-domain placement for queueing delay; it is not a tail-latency fix.

Index Terms: *CPU scheduling, hybrid processors, tail latency, EEVDF, Thread Director, wakeup latency, performance isolation.*

I. Introduction

Hybrid CPUs are now common in laptops and small desktops. They combine performance (P) and efficiency (E) cores on one die [1]. Intel's Thread Director supplies the scheduler with core-type and performance hints for each thread [1], [2], but Linux still chooses the final CPU in the wakeup path. A 13th Gen Intel Core i7-13650HX pairs six P-cores with eight E-cores [3]. The P-cores have higher peak frequency and SMT; the E-cores have lower frequency and no SMT. Linux treats both core types as CPUs in one scheduling domain. EEVDF selects the next task by virtual deadline [4]–[6], while the wakeup path and load balancer decide which CPU receives a waking task. EEVDF replaced CFS as the default general-purpose policy in Linux 6.6 [4], and a 2024 evaluation reports lower latency for 5G control-plane workloads under EEVDF [7]. For batch throughput, a poor placement costs energy. For an interactive task, a poor placement costs a visible stall.

Tail latency is the right metric for this problem. Scheduler policy already changes the tail for latency-sensitive multithreaded services [8], and at microsecond scale the scheduler itself becomes the bottleneck [9]. A 43 µs median wakeup is invisible; a 2.8 ms wakeup is heavily noticeable. The p99.9 is not an abstract percentile: with 120 wakeups per 15 s run, it is the worst wakeup or two in that run. We therefore treat the per-run p99.9 as the unit of analysis and ask whether pinning a latency-sensitive workload to E-cores makes that worst case predictable.

Hybrid CPUs are a case of asymmetric multicore. Asymmetry-aware policies outperform default Linux placement on Alder Lake [10], building on a long line of capacity-aware OS scheduling [11] and a broad design space [12]. Static affinity is the simplest such policy. It still relies on the load balancer within the mask, and Linux load balancing has a documented history of work-conservation failures on older kernels [13], which we treat as precedent rather than as evidence about this kernel. One might therefore expect practitioners to pin interactive tasks to E-cores. The reasoning is simple: if the P-cores are saturated by batch work, an E-core is isolated from that contention. The assumption is that isolation buys a lower tail. We test the assumption under a controlled mixed load. We saturate all 12 P-core SMT siblings with sysbench and run schbench with eight workers (two message threads of four workers each). Exp A lets schbench float across CPUs 0–17. Exp B pins it to CPUs 12–17, the six-E-core test pool. Each arm has 30 runs. We record sched_wakeup, sched_switch, and sched_migrate_task for every run, and we reconstruct the runqueue state around slow wakeups from a separate 30 s trace.

The results are not a simple ranking. The body of the distribution is identical: 43 µs median floating, 44 µs pinned. The per-run p99.9 median is 162 µs floating and 701 µs pinned, a 4.3x gap, but the distributions cross. Five floating runs and four pinned runs exceed 1 ms; the maxima are 2836 µs and 2852 µs. A paired Wilcoxon test gives $p = 0.17$, and the bootstrap 95% CI for the median difference spans zero. The data support a mechanism and a repeatable descriptive pattern, *not a universal claim that floating is faster*.

The traces identify the mechanism. With the P-cores loaded, 0 of 702 floating wakeups target a P-core; the baseline without load sends 541 of 730 wakeups to P-cores. Floating is E-pool by default, with rare cross-domain migrations. The multi-millisecond tail comes from wakeup-window collisions: a target E-core looks idle to two wakeups a few microseconds apart, both enqueue, and the loser waits for the winner's request. In the 707-wakeup trace, 61 wakeups (8.6%) hit an occupied target; all nine wakeups at or above 100 µs had a collision, and four were placement races. The trace also shows 602 windows in which a runnable schbench thread waited while an E-core was idle, totaling 10.6 ms over 30 s.

We make four contributions. First, we provide a paired $N = 30$ wakeup-tail dataset for floating and E-pool pinning on a hybrid Intel CPU under P-core saturation. Second, we quantify the tail with both per-run percentiles and trace-level collisions. Third, we show that E-core pinning raises the typical tail and leaves the multi-millisecond worst case intact. Fourth, we give a reproducible measurement harness and a trace parser that separate placement races from raw E-core slowness.

II. Experimental Environment and Methodology

A. Testbed and topology

All measurements come from one machine: an Intel Core i7-13650HX [3] (Raptor Lake, six P-cores, eight E-cores, 24 MB L3). The kernel is 7.1.8-arch1-3 with EEVDF. The P-cores expose SMT siblings as CPUs 0–11; the E-cores are CPUs 12–19. P-core

maximum frequency is 4.7–4.9 GHz; E-core maximum is 3.6 GHz.

The harness masks cover CPUs 0–17. CPUs 18 and 19 are outside both affinity sets - for housekeeping. The E-pool in this paper is therefore six of the eight physical E-cores. This matches the original rig's hardcoded masks, but it reduces the pinned pool's capacity and is a limitation, not a property of the SKU.

Table 1 lists the testbed and workload parameters. The 30-run sweep recorded the powersave governor in environment_info.txt. A separate 30 s trace capture for mechanism analysis ran under the performance governor. We do not compare absolute latencies across those two sets; the trace is used only for event-level reconstruction.

Table 1. Testbed and workload parameters.

Parameter	Value
CPU	Intel Core i7-13650HX: 6 P-cores (CPUs 0–11, SMT), 8 E-cores (CPUs 12–19)
Kernel	7.1.8-arch1-3, EEVDF
Governor	powersave (N = 30); performance (30 s trace)
Background	sysbench cpu, 12 threads, max-prime 50M, CPUs 0–11
Probe	schbench -m 2 -t 4 -r 15 -s 1000000
Runs	30 paired, 5 s warmup, 15 s measure, 2 s cooldown
Tracing	sched_switch, sched_wakeup, sched_migrate_task
Affinity	taskset only; no cgroup cpuset

B. Background load and probe

The background load is sysbench cpu --threads=12 --cpu-max-prime=50000000, pinned to CPUs 0–11 [14]. Twelve threads place two runnable threads on each P-core, keeping both SMT siblings busy. Each iteration starts sysbench, waits 5 s, runs the probe, kills sysbench, and waits 2 s.

The probe is schbench -m 2 -t 4 -r 15 -s 1000000: two message threads, four workers each (eight workers total), 15s runtime, and a 1s sleep between requests [15]. The sleep is why request latencies are about 1,004,544 µs; we ignore the Request Latencies block. Each run produces 120 wakeup samples. We parse the final Wakeup Latencies block, which covers the full 15s.

Every probe run is wrapped in perf record -e sched:sched_switch,sched:sched_wakeup,sched:sched_migrate_task. The capture writes about 0.1 MB per run and contains several hundred samples (571 in run 1). perf script produces the text stream used by the parser. The trace adds overhead; both arms carry it, but the pinned arm has fewer CPUs to absorb it. Section V treats this as a measurement factor.

C. Experimental arms

Exp A (floating) runs `taskset -c 0-17 schbench -m 2 -t 4 -r 15 -s 1000000`. The probe can use all 18 CPUs in the mask. Exp B (pinned E-pool) runs the same probe under taskset -c 12-17. The probe is confined to six E-cores.

Each of 30 iterations runs A then B. A always precedes B; we did not randomize order. A 2s cooldown separates the arms. Both workloads run at normal priority. No cgroup cpuset controller, CPU isolation, or real-time priority is used in the recorded runs. Affinity is enforced with taskset only.

D. Metrics, data processing, and statistics

The unit of analysis is the per-run p99.9 wakeup latency. The schbench log reports percentiles over 120 samples, so p99.9 is effectively the maximum. In 27/30 floating runs and 21/30 pinned runs, p99.9 equals the reported max. We treat it as a tail indicator, not a stable percentile. This is deliberate.

We summarize each arm with median, mean, standard deviation, min, max, and counts above thresholds. Because A and B are paired by iteration, we use a two-sided Wilcoxon signed-rank test. We also report a 20,000-resample bootstrap 95% confidence interval for the difference in medians and means, and Fisher exact tests for threshold counts.

The harness CSV contains wrapped and duplicated rows. We rebuild the table from the raw 30 logs with analysis/parse_results.py. All numbers in this paper come from the rebuilt analysis/distribution_data.csv. The 15 s block is the last Wakeup Latencies block in each log. Artifact: the repository contains the harness, the cleaned per-run table, the summary statistics, the trace parser, and the figure scripts.

E. Trace reconstruction

We use a separate 30s floating run under the performance governor, captured with the same three tracepoints. The parser walks wakeup, switch, and migrate events in timestamp order and maintains a per-CPU runqueue set. A wakeup adds the task to the target CPU; a switch removes the next task; a switch with prev_state=R leaves the previous task queued. This is an approximation, not perf sched.

A collision is a wakeup whose target CPU is not idle, either running another task or holding a queued wakee. A placement race is a collision where the target looked idle in the switch stream but a previously woken task had not run. An idle-core mismatch is an interval with at least one runnable schbench thread and at least one E-core idle with an empty queue. The trace contains 707 matched schbench wakeup-to-first-run pairs. It does not contain newidle_balance tracepoints; Section IV is explicit about what that prevents us from claiming. Four captures feed the trace analysis in this paper: the phase 1 baseline with no background load (730 schbench wakeups), the phase 1 floating run (702), and the phase 1 pinned run (719), all used only for wakeup-target and migration counts and collected without recording a governor; and the 30 s phase 1.1 floating capture under the performance governor (707 matched wakeups), which supplies every collision and idle-mismatch number below.

III. Empirical Results

A. The body of the distribution is unchanged

Table 2 gives the wakeup latency summary. Normal wakeups are the same in both arms: median p50 is 43 µs floating and 44 µs pinned, and median p90 is 71 µs and 64.5 µs. The pinned p90 is 6.5 µs lower, which is within run-to-run movement and does not favor a claim. The p99 medians are 137 µs and 139 µs. The difference is in the top 0.1%.

Table 2. Wakeup latency summary (µs), N = 30 per arm. p99.9 rows use the 30 per-run values; p50–p99 rows report the median of the 30 per-run percentiles.

Metric	Exp A floating	Exp B E-pool
p50 median (min–max)	43 (39–49)	44 (7–46)
p90 median (min–max)	71 (53–85)	64.5 (51–83)
p99 median (min–max)	137 (68–827)	139 (89–2276)
p99.9 mean	527.8	688.2
p99.9 median	162	701
p99.9 standard deviation	621.7	628.4
p99.9 min–max	79–2836	119–2852
> 250 µs	13/30	19/30
> 500 µs	12/30	18/30
> 1 ms	5/30	4/30
> 2 ms	1/30	2/30

Figure 1 plots the ECDF of the 30 per-run p99.9 values. Through roughly the 80th percentile of runs, the floating curve lies to the left of the pinned curve. The median gap is 539 µs (162 vs

701). Above 1 ms the curves cross: floating has five runs above 1 ms, pinned has four, and the pinned maximum is 16 µs higher (2852 vs 2836). The curves cross because floating has more runs between 1 and 2.8 ms, while pinned has more runs between 250 and 1000 µs.

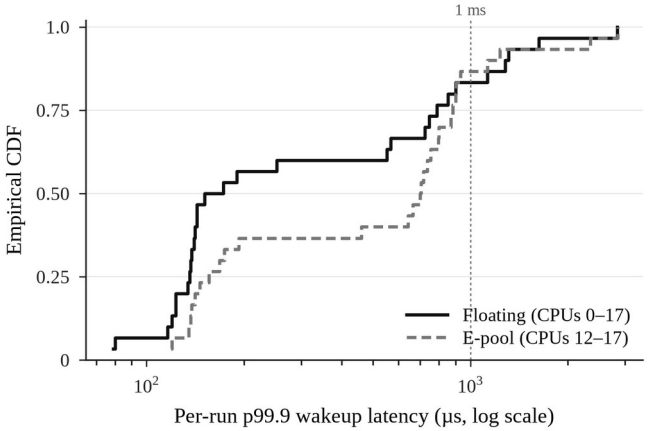


Figure 1. ECDF of the 30 per-run p99.9 wakeup latencies. The floating curve is lower for most runs, but the curves cross near 1 ms and both arms contain multi-millisecond events. The dashed vertical line marks 1 ms.

Figure 2 shows the same data as a box-and-whisker plot with individual runs. The floating distribution is bimodal: a dense low cluster around 120–200 µs and a sparse high cluster around 700–1300 µs. The pinned distribution keeps a low shoulder but moves most of its mass into 600–900 µs. The box plot makes the trade explicit: pinning removes many of the best runs but not the worst.

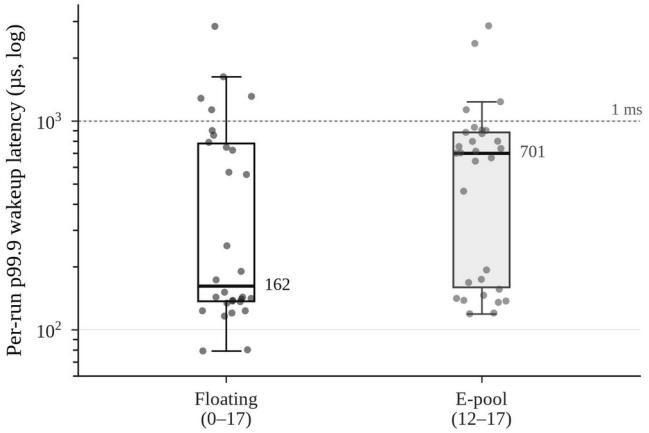


Figure 2. Box-and-whisker plot with individual runs, log scale. Pinning raises the median from 162 µs to 701 µs but leaves both arms with points above 2.8 ms.

B. Pinning relocates the tail

Pinning raises the median p99.9 by 4.3× (701 vs 162) and the mean by 160 µs (688.2 vs 527.8). The mean gap is smaller than the median gap because floating's mean is inflated by a 2836 µs run; both standard deviations are about 625 µs. The medians are a better summary of the typical run, but the means show that both arms carry substantial extreme mass.

The threshold counts separate the bulk from the extreme. Pinning raises the count above 250 µs from 13/30 to 19/30 and the count above 500 µs from 12/30 to 18/30. It does not reduce the count above 1 ms: 4/30 pinned vs 5/30 floating. It has two runs above 2 ms (2348 and 2852 µs) vs one floating (2836 µs). Fisher exact tests give p = 0.20 for >250 µs, p = 0.20 for >500 µs, and p = 1.0 for >1 ms. The direction of the bulk shift is consistent; the extreme tail is not improved.

Run 7 in the pinned arm is not a single bad sample. Its p99 is 2276 µs and its p99.9 is 2348 µs, with ten samples at the 99th percentile. The 2852 µs case is run 15. In the floating arm, run 12 is the 2836 µs case; its p99 is 135 µs, so the event is isolated. Pinning changes the shape of the tail: more mid-tail runs, plus rare long stalls.

C. The distributions cross, and the ranking is uncertain

Sixteen of 30 pinned runs have a higher p99.9 than their paired floating run; 14 have a lower one. The probability that a randomly chosen pinned run is worse than a randomly chosen floating run is 0.62. That is a weak separation. The median gap comes from the bulk, not the extremes: 17 floating runs sit below 200 µs, versus 11 pinned runs. Pinned is more than 500 µs worse in 13 pairs, while floating is more than 500 µs worse in five. The two largest absolute differences are floating losses: run 12 (2836 vs 737 µs) and run 14 (1626 vs 156 µs). Those losses inflate floating's extreme tail but do not move its median.

The paired Wilcoxon signed-rank test gives W = 166.5, p = 0.175. The median difference is 539 µs, with a bootstrap 95% CI of −58 to +646 µs. The mean difference is 160.4 µs, with a CI of −149 to +471 µs. The CIs include zero. At N = 30 and with standard deviations above 600 µs, the experiment cannot resolve a 4.3x median gap into a statistically reliable ordering. We report the gap because it is large and because the trace explains where it comes from.

This is why we do not claim that floating is faster in general. The defensible claim is conditional: in this workload on this machine, floating has a lower typical tail, and the difference is produced by a small number of mid-tail runs. Pinning does not cap the worst case.

D. What the trace adds

The distribution answers how much, not why. The 30s trace answers why. Over 707 matched schbench wakeups, the wakeup-to-first-run delay is p50 2.0 µs, p90 15.0 µs, p99 188.5 µs, p99.9 2807 µs, and max 2812 µs. These are all wakeups, including ones schbench does not sample, so they are not directly comparable to the per-run schbench p99.9. They show the same shape: a microsecond body and a multi-millisecond tail.

Sixty-one of 707 wakeups (8.6%) hit a target CPU that was not idle. In 54, the target was running another task; in 7, it held a queued wakee that had not run. All nine wakeups at or above 100 µs had a target collision. Four of the nine were placement races: the target looked idle in the switch stream, but another task had already been woken to it. Thirteen wakeups (1.8%) chose a busy E-core while a different E-core was idle. The runqueue model finds 602 idle-core mismatch intervals, 10.6 ms in total over 30s (0.035% of the trace), with a longest interval of 3.76 ms. Table 3 summarizes the counts.

Table 3. Trace-level mechanism counts. One 30 s floating capture, 707 matched schbench wakeups.

Observation	Value
Matched wakeups	707
p50 / p90 / p99 / p99.9	2.0 / 15.0 / 188.5 / 2807 µs
Maximum	2812 µs
Target occupied at wakeup	61 (8.6%)
— target running another task	54
— target had pending wakee	7
Slow wakeups ≥ 100 µs with collision	9/9
— placement races	4/9
Busy E-core chosen while another idle	13 (1.8%)
Idle-core mismatch intervals	602
Total mismatch time / 30 s	10.6 ms
Longest mismatch	3.76 ms

IV. Root-Cause Analysis

A. Wakeup-window collisions

The worst event is the clean example. At t = 119.828122, a schbench worker (pid 834) is woken and placed on CPU 15, which is idle. At t = 119.828126, four microseconds later, a second worker (pid 833) is also placed on CPU 15. The target still looks idle to the wakeup path because pid 834 has not switched in yet. Pid 834 switches in at t = 119.828129 and runs a full request until t = 119.830933. Pid 833 waits in the runqueue until t = 119.830938, 2812 µs after its wakeup. The same pattern appears in the 2805 µs event on CPU 17.

Figure 3 shows the timeline. The collision is not an E-core executing slowly. The first worker runs for 2.8 ms; the second worker pays for that entire slice. The scheduler's target selection sees a snapshot: CPU 15 is idle at both wakeup instants. The enqueue and the switch-in are not atomic with respect to another wakeup. Two wakeups can therefore select the same just-idle CPU within the wakeup window.

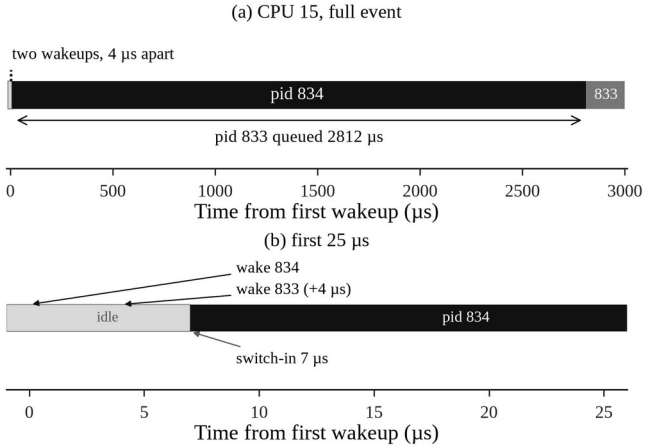


Figure 3. The 2812 µs collision in the 30 s floating trace. Both wakeups select CPU 15 while it is idle; the first switch-in occurs 7 µs after the first wakeup, and the second worker waits for the first worker's full request.

The window is 4 µs short in this event, but the cost is not. EEVDF gives the first task a virtual deadline and a slice; the loser waits for the winner to yield, migrate, or be preempted. With eight workers and two message threads on six E-cores, there is little slack to hide the wait. The 707-wakeup trace has nine events at or above 100 µs; all nine hit an occupied target. Collisions are necessary but not sufficient: of the 61 wakeups that hit an occupied target, only these nine waited past 100 µs, because a collision costs time only when the winner holds the CPU for a long request. The 4 µs race between the two wakeups is the mechanism itself, not a correlate; the second worker could not have been slow for any other reason.

B. Work-conservation deficit

A second mechanism shows up as idle-core mismatch. In 13 of 707 wakeups, the scheduler chose a busy E-core while another E-core was idle. The runqueue model finds 602 intervals in which at least one schbench thread was runnable and at least one E-core was idle with an empty queue. The total is 10.6 ms over 30s; the longest is 3.76 ms, with a thread queued on CPU 17 and CPU 14 idle. That interval outlasts the longest wakeup-to-first-run delay (2.8 ms) because it starts from a thread that was already runnable - a preempted or requeued task - rather than a single fresh wakeup, so it is not bounded by the wakeup delay.

This is a work-conservation deficit at microsecond resolution. The scheduler is work-conserving on paper: if a CPU is idle and another runqueue has runnable work, a balancing path should move work. Work-conservation failures have been documented before, on much older CFS kernels [13]; we take that as precedent for the possibility, not as proof about 7.1.8. The trace shows windows where it does not happen fast enough to matter for the tail. The likely path is a bounded idle search and deferred balancing: select_idle_sibling() and its helpers scan a limited set of CPUs, and newidle_balance() pulls work only when a CPU is about to go idle. If the search budget is exhausted or the balance is skipped, an idle E-core can sit unused while a wakee queues elsewhere.

The trace set does not include newidle_balance tracepoints, so we cannot count aborts or attribute each mismatch to a specific bailout. We can measure the consequence: 602 windows, 10.6 ms, 3.76 ms longest. The mechanism is a hypothesis consistent with the runqueue state; the deficit itself is measured. A trace with sched_balance tracepoints or perf sched is the next step.

C. Why the hybrid topology amplifies both

Two properties of the topology make both mechanisms worse. First, the P-cores are effectively removed from wakeup placement while loaded. In the phase 1 capture, the idle baseline sends 541 of 730 schbench wakeups to P-cores. With sysbench on CPUs 0–11, the floating run sends 0 of 702 to P-cores, and the pinned run sends 0 of 719. This is idle-CPU search, not capacity-aware scheduling: with all twelve P-lanes busy, the idle CPUs the wakeup path finds are E-cores. Floating is therefore E-pool by default; in the captured run, the observed route to a P-core is a load-balancer migration. The phase 1 floating capture has one E to P and one P to E migration out of 119 schbench migrations. The pinned capture has 70 migrations, all within the E-pool. The lower floating median coincides with this rare cross-domain capacity, but the traces do not prove that every low run used a P-core.

Second, the E-pool is small: six E-cores for eight workers, two message threads, and kernel work. At the worst collision, all other E-cores were busy. There is no spare E-core to absorb the loser. The wakeup window is microseconds, but the request is milliseconds; a single collision can dominate a 15s run's p99.9. This is why the pinned arm's p99.9 median rises while its p50 and p90 stay in the same range.

D. Alternative explanations

Three other explanations are plausible and not excluded by this data. First, E-cores are slower per core; a queued request takes longer to drain. The trace shows the loser waits for the winner's full request, which is a capacity effect as well as a placement effect. Second, C-state exit latency and IRQ steering can add

hundreds of microseconds to a wakeup; the ~700 µs pinned median may include time that is not scheduler queueing. Third, perf record adds overhead, and the pinned arm has fewer cores to absorb it. We did not measure per-run frequency, C-state residency, IRQ affinity, or perf overhead. Section V treats these as limitations. The collision mechanism is directly observed; the share of the median gap caused by each factor is not.

V. Limitations

Single machine, single kernel. The data come from one i7-13650HX, one 7.1.8-arch1-3 kernel, and one EEVDF configuration. Different P/E layouts, cache sizes, or scheduler tunings can change the balance. The results are a case study, not a law of nature.

Governor and tracing. The N = 30 sweep recorded powersave, not performance. Every run was inside perf record. The pinned arm has fewer cores to absorb tracing overhead, so the absolute gap may narrow without tracing. We do not treat 162 vs 701 as a universal constant. The trace deep-dive ran under performance, so its absolute latencies are not directly comparable to the N = 30 table.

Sample size and metric. There are 30 runs per arm and 120 wakeups per run. p99.9 is effectively the maximum; 27/30 floating and 21/30 pinned runs have p99.9 equal to max. A longer run with more samples would estimate p99.9 more precisely. The paired tests are underpowered for the observed variance; the confidence intervals include zero.

Topology coverage. The E-pool is CPUs 12–17, six of the eight E-cores. CPUs 18–19 are outside both masks. The pinned arm therefore runs on a smaller pool than the SKU provides. This likely raises the pinned baseline; it does not explain the collisions, which occur within the tested pool.

Workload. schbench with eight workers and a 1s sleep is synthetic. It stresses wakeup placement, but it is not a browser, audio server, or game. The fixed 1 s sleep also means request latency is not informative. A real interactive workload may have different queue depths and blocking behavior.

Trace reconstruction. The parser uses wakeup, switch, and migrate events only. It approximates runqueue state and can miss wakeups that are not followed by a switch in the capture window. It is not perf sched. The 30 s trace is a single floating run, not 30 runs; its mechanism frequencies are illustrative, not a distribution.

Missing instrumentation. There are no newidle_balance, select_idle_sibling, C-state, IRQ, or frequency tracepoints. We measure the work-conservation deficit but cannot attribute it to a specific balance abort. There is no per-run thermal or frequency telemetry, no cgroup cpuset, and no real-time priority or uclamp hint.

Order effects. A always precedes B. The 2 s cooldown reduces but does not eliminate drift. A randomized order would be better. These limits bound the effect size, not the mechanism. The 4 µs collision and the 2812 µs wait are direct observations.

VI. Conclusion and Future Work

We measured wakeup tail latency on a hybrid Intel CPU under P-core saturation. Floating and E-pool pinning produce the same 43–44 µs median wakeup. They differ in the tail: the per-run p99.9 median is 162 µs floating and 701 µs pinned, but 5/30 floating and 4/30 pinned runs exceed 1 ms, and the maxima are 2836 and 2852 µs. Pinning moves mass into the 250–1000 µs band; it does not remove multi-millisecond stalls. The paired test does not establish a reliable ranking (p = 0.17), because the distributions cross.

The trace explains the stalls. With the P-cores loaded, 0/702 floating wakeups target a P-core. The multi-millisecond tail comes from wakeup-window collisions: two wakeups select the same just-idle E-core within 4 µs; the loser waits 2812 µs for the winner's request. All nine wakeups at or above 100 µs hit an occupied target, and four were placement races. The trace also shows 602 windows of idle-core mismatch, 10.6 ms over 30 s, longest 3.76 ms. The scheduler is work-conserving on paper, but the wakeup path and the balancing path do not close the gap fast enough at this scale.

Blanket E-core pinning is not a tail-latency fix. It trades a rare cross-domain placement for queueing delay and leaves the worst case intact. Asymmetry-aware policies outperform default Linux placement on Alder Lake [10], with earlier capacity-aware scheduling as precedent [11]; a better policy would reserve a small P-core slice for the latency-critical task, or use uclamp and cpuset to give it capacity without making it share a six-core pool with all other work. A queue-depth-aware wakeup path could also avoid sending a second wakee to a core that already has a pending wakee.

Future work has four parts. First, rerun N = 30 under the performance governor without perf record, with randomized arm order and longer runs, so p99.9 is a real percentile rather than a max. Second, instrument newidle_balance, select_idle_sibling, C-state exit, IRQ affinity, and frequency, to split the ~700 µs pinned baseline into scheduler and platform components. Third, test all eight E-cores and a reserved P-core slice, including cgroup cpuset and uclamp variants. Fourth, try a scheduler change that makes the wakeup path aware of pending wakees, or that weighs queue depth against core capacity, and measure whether it removes the 2.8 ms collisions without hurting throughput. The same rig can be repeated on other hybrid SKUs to see which findings are Intel-specific.

References

[1] E. Rotem et al., “Intel Alder Lake CPU architectures,” IEEE Micro, vol. 42, no. 3, pp. 13–19, May/Jun. 2022, doi: 10.1109/MM.2022.3164338.

[2] J. C. Sáez and M. Prieto-Matias, “Evaluation of the Intel Thread Director technology on an Alder Lake processor,” in Proc. 13th ACM SIGOPS Asia-Pacific Workshop Syst. (APSys), 2022, pp. 61–67, doi: 10.1145/3546591.3547532.

[3] Intel Corporation, “Intel Core i7-13650HX processor specifications,” 2023. [Online]. Available: https://ark.intel.com/content/www/us/en/ark/products/232101/

[4] Linux Kernel Documentation, “EEVDF scheduler,” Documentation/scheduler/sched-eevdf.rst. [Online]. Available: https://docs.kernel.org/scheduler/sched-eevdf.html

[5] I. Stoica and H. Abdel-Wahab, “Earliest eligible virtual deadline first: A flexible and accurate mechanism for proportional share resource allocation,” Dept. Comput. Sci., Old Dominion Univ., Norfolk, VA, USA, Tech. Rep. TR-95-22, 1995.

[6] J. Corbet, “An EEVDF CPU scheduler for Linux,” LWN.net, 2023. [Online]. Available: https://lwn.net/Articles/925371/

[7] M. Impesi, “Evaluation of Linux scheduler algorithms for low latency control plane applications,” M.S. thesis, Dept. Comput. Inf. Sci., Linköping Univ., Linköping, Sweden, 2024. [Online]. Available: https://www.diva-portal.org/smash/record.jsf?pid=diva2%3A1884324

[8] E. Asyabi, E. Sharafzadeh, S. A. SanaeeKohroudi, and M. Sharifi, “CTS: An operating system CPU scheduler to mitigate tail latency for latency-sensitive multi-threaded applications,” J. Parallel Distrib. Comput., vol. 133, pp. 232–243, Nov. 2019, doi: 10.1016/j.jpdc.2018.04.003.

[9] K. Kaffes, T. Chong, J. T. Humphries, A. Belay, D. Mazières, and C. Kozyrakis, “Shinjuku: Preemptive scheduling for µsecond-scale tail latency,” in Proc. 16th USENIX Symp. Netw. Syst. Design Implement. (NSDI), 2019, pp. 345–360.

[10] C. Bilbao, J. C. Sáez, and M. Prieto-Matias, “Flexible system software scheduling for asymmetric multicore systems with PMCSched: A case

for Intel Alder Lake," Concurrency Comput., Pract. Exp., vol. 35, no. 25, e7814, 2023, doi: 10.1002/cpe.7814.

[11] T. Li, D. Baumberger, D. A. Koufaty, and S. Hahn, "Efficient operating system scheduling for performance-asymmetric multi-core architectures," in Proc. ACM/IEEE Conf. Supercomput. (SC), 2007, pp. 1–11, doi: 10.1145/1362622.1362694.

[12] S. Mittal, "A survey of techniques for architecting and managing asymmetric multicore processors," ACM Comput. Surv., vol. 48, no. 3, art. 45, pp. 1–38, 2016, doi: 10.1145/2856125.

[13] J.-P. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma, and A. Fedorova, "The Linux scheduler: A decade of wasted cores," in Proc. 11th Eur. Conf. Comput. Syst. (EuroSys), 2016, pp. 1–16, doi: 10.1145/2901318.2901326.

[14] A. Kopytov, "sysbench," GitHub repository. [Online]. Available: https://github.com/akopytov/sysbench

[15] C. Mason, "schbench," GitHub repository. [Online]. Available: https://github.com/masoncl/schbench

[16] S. Vattikuti, "Tail Wakeup Latency on Intel Hybrid CPUs: Evaluating Linux CPU Placement and Wakeup Placement Under Mixed Load," Zenodo, Oct. 2026, doi: 10.5281/zenodo.23674242.

- Reference GitHub repository: https://github.com/woterr/hybrid-eevdf-latency-analysis

Appendix A. Per-Run Measurements

Table A1 lists the 30 paired p99.9 values in collection order. The paired analysis in Section III.C uses these values. Exp A is floating (CPUs 0–17); Exp B is pinned to the six-E-core pool (CPUs 12–17).

Table A1. Per-run p99.9 wakeup latency (µs).

Run	A	B	Run	A	B
1	723	174	16	567	120
2	553	168	17	1310	703
3	136	141	18	190	146
4	899	881	19	137	905
5	143	665	20	141	1234
6	123	1130	21	1130	931
7	1282	2348	22	789	193
8	143	699	23	120	753
9	140	135	24	79	715
10	151	138	25	123	797
11	173	119	26	134	460
12	2836	737	27	116	799
13	853	869	28	80	641
14	1626	156	29	138	137
15	747	2852	30	252	899